USER MANUAL · V2.0

SGLottery

Singapore 4D & TOTO Ticket Scanner Application

VERSION	DATE	PLATFORM
2.0	March 2026	Web · Mobile (Android/iOS)

STATUS

Prototype / Assessment Build

📸 OCR Ticket Scanning 🔔 Push Notifications 📊 Past Results 🔮 Predictive Analysis

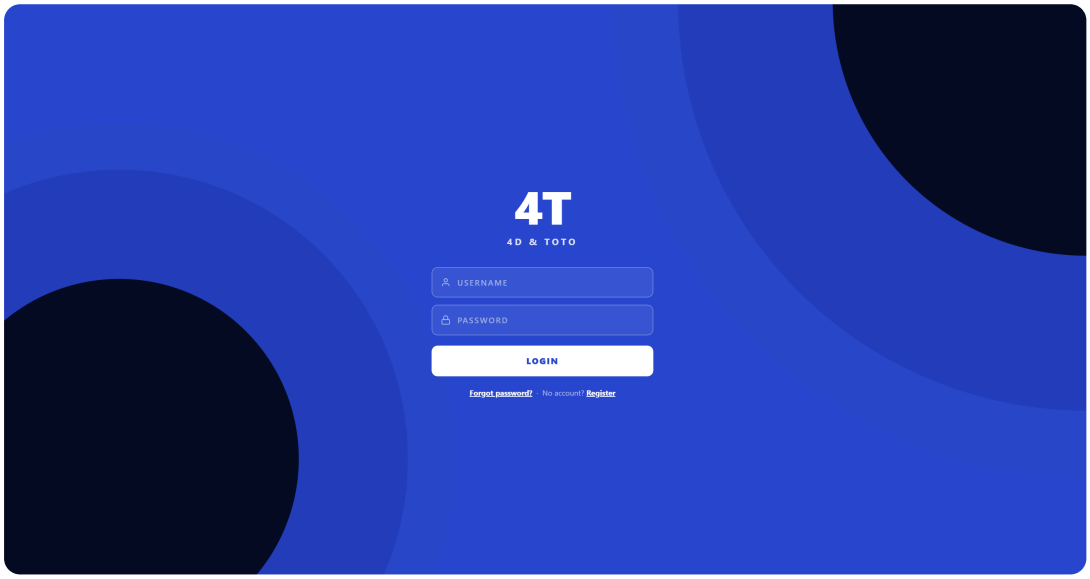
📋 Ticket History 🎯 TOTO System Bets

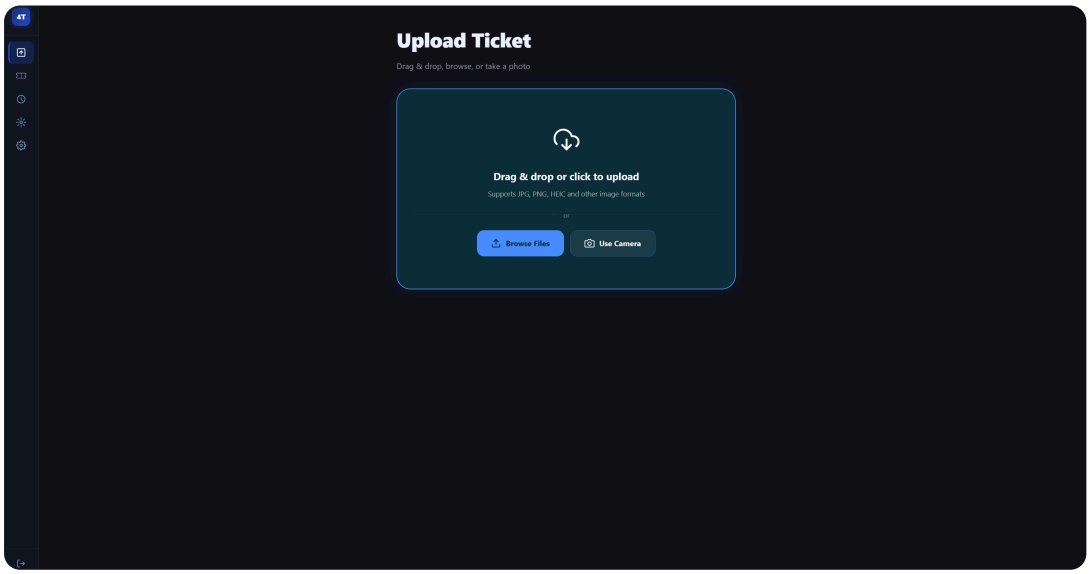
Table of Contents

1 Overview

SGLottery is a full-stack application for scanning, storing, and analysing Singapore Pools 4D and TOTO lottery tickets. It covers the complete ticket lifecycle — from photo upload and OCR extraction, through automated result checking, to predictive statistical analysis.

The application requires a user account. On first visit, users are presented with a **Login / Register** screen. After authenticating, they are taken to the main dashboard with full access to all features.





Key Features

Feature	Description
📸 OCR Ticket Scanning	Upload a photo of any Singapore Pools 4D or TOTO ticket; numbers, dates, bet type and serial number are extracted automatically using OCR.
🎯 TOTO System Bet Expansion	System 7–12 tickets are automatically expanded into all C(N,6) combinations (up to 924 for System 12) and stored in Firestore.
✅ Automated Result Checking	Past-draw tickets are checked immediately on upload. Future-draw tickets are polled hourly by a backend cron job.
🔔 Push Notifications	Win/loss alerts delivered via browser push (web) and Expo push notifications (mobile) when results become available.
🔮 Predictive Analysis	Three independent statistical models generate 4D number and TOTO System 12 predictions from historical draw data.
📊 Past Results Browser	Scrapes Singapore Pools for 4D and TOTO past results with mock-data fallback if scraping is blocked.
📋 Ticket History	Full searchable, filterable, sortable history of all scanned tickets with detail modal and inline result checking.

Platform Support

Platform	Status	Notes
Web (Desktop Browser)	✅ **Full**	Chrome, Firefox, Edge — http://localhost:5173
Mobile Web (Browser)	✅ **Responsive**	Same URL, fully responsive layout
Android (Expo Go)	✅ **Full**	Scan QR code with Expo Go app
iOS (Expo Go)	✅ **Full**	Scan QR code with Expo Go app

> **Assessment Note**
> This manual is intended for evaluators and future interns. All steps assume a fresh machine with no prior project setup.

2 Prerequisites & Dependencies

Required Software

Tool	Minimum Version	Installation
Node.js	18.x or later	https://nodejs.org — download LTS installer
npm	9.x or later	Included automatically with Node.js
Git	2.x	https://git-scm.com
Expo Go App	Latest	iOS App Store · Google Play Store

Optional — For Emulator Testing

Tool	Purpose
Android Studio	Android emulator (not required for physical device)
Xcode 15+ (macOS only)	iOS simulator

External Services

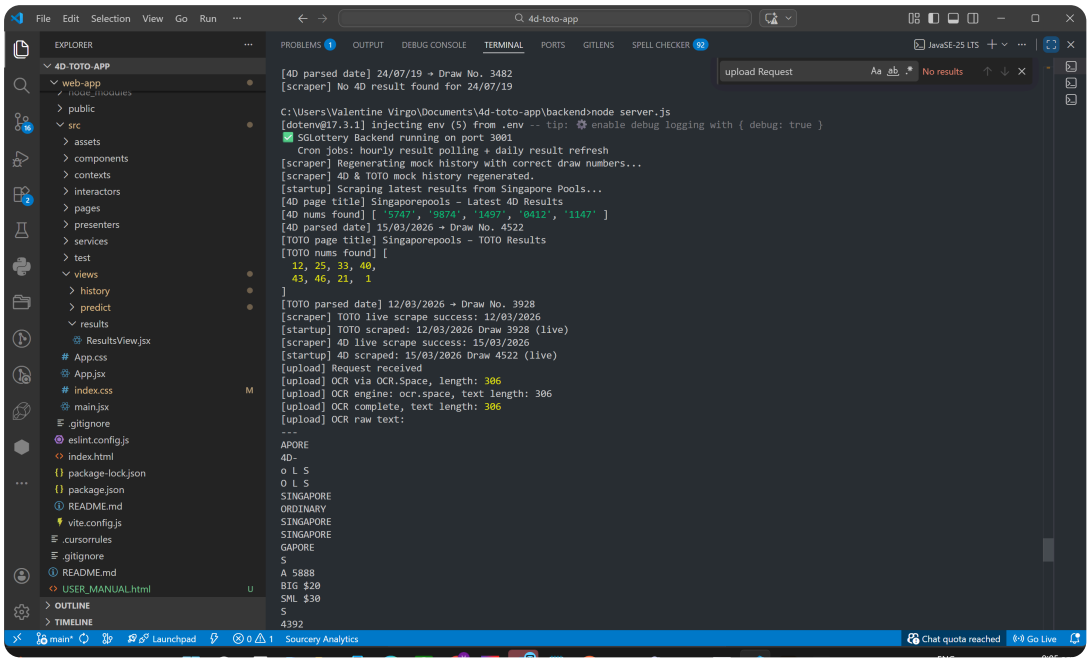
Service	Purpose	Free Tier
Firebase (Firestore)	Database — stores tickets and draw results	Yes — Spark plan
ngrok	Expose localhost to physical mobile device	Yes — 1 tunnel free
OCR.Space API	Primary OCR engine (fallback to Tesseract if unavailable)	Yes — free key included

> ⚠️ **Firebase Storage**
> Image thumbnail storage requires the Firebase Blaze (pay-as-you-go) plan. In this prototype build, image storage is disabled — all other features work fully on the free Spark plan.

Verify Your Node.js Installation

Open a terminal and run:

```
node --version   # Should show v18.x or higher
npm --version    # Should show 9.x or higher
```



3 Installation & Setup

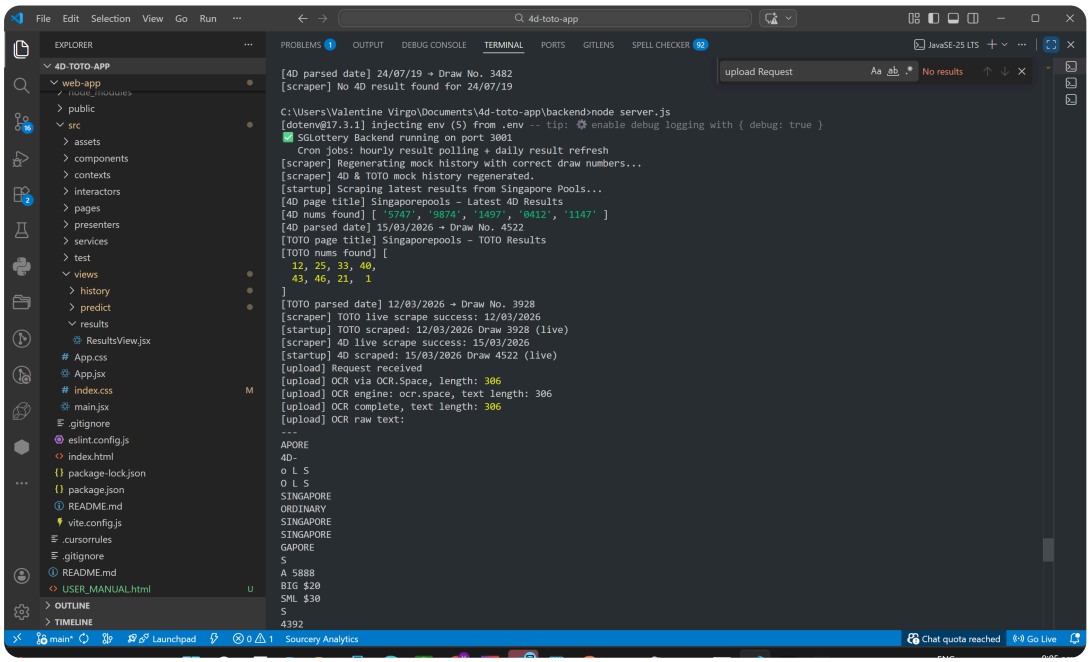
3.1 Clone the Repository

```
git clone <your-repository-url>
cd 4d-toto-app
```

3.2 Install Dependencies

Each sub-project has its own dependencies. Install all three:

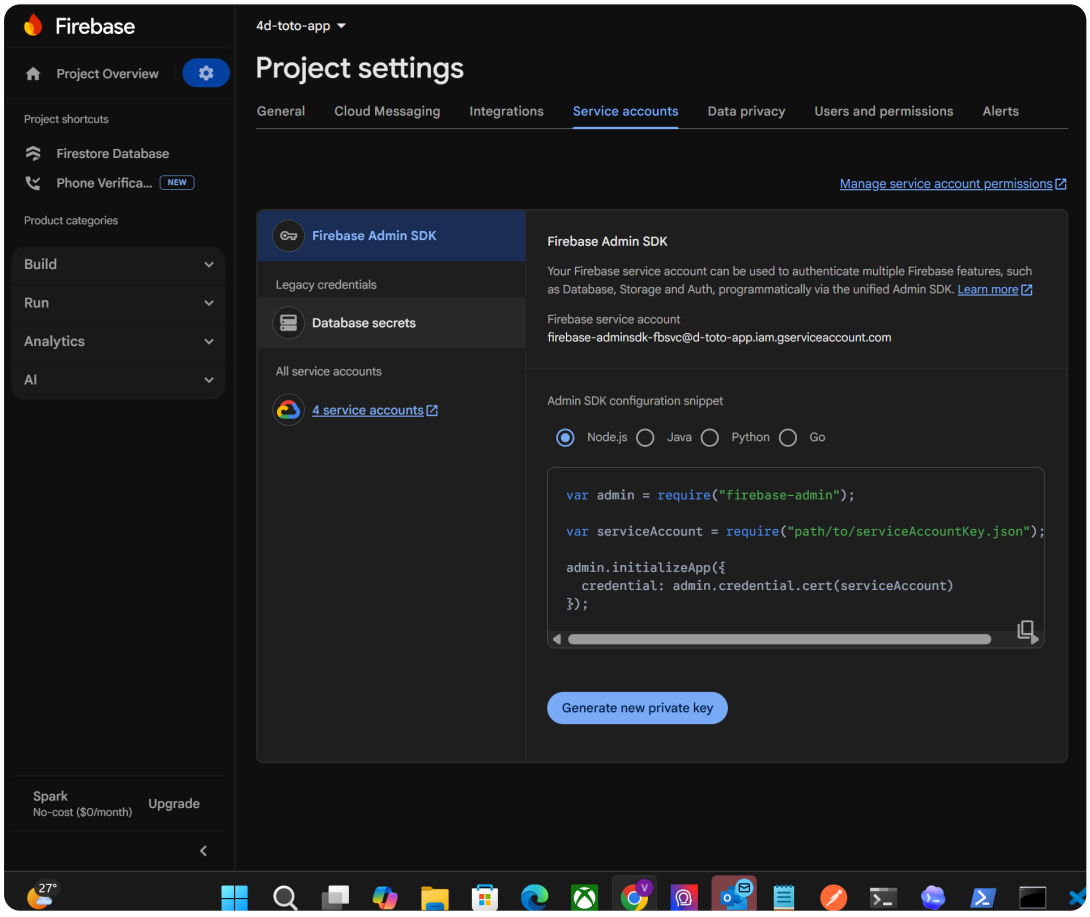
```
# Backend
cd backend
npm install

# Web App
cd ../web-app
npm install

# Mobile App
cd ../mobile-app
npm install
```



3.3 Firebase Setup

1. Go to **https://console.firebase.google.com** and sign in with your Google account.
2. Select your existing project `d-toto-app` or click **Add Project** to create a new one.
3. In the left sidebar, click **Firestore Database** → **Create Database** → choose **Native mode** → select any region → click **Enable**.
4. Go to **Project Settings** (gear icon, top-left) → **Service Accounts** tab.
5. Click **Generate New Private Key** → confirm → save the downloaded JSON file as `backend/serviceAccountKey.json`.

> **Security Warning**
>
> The `serviceAccountKey.json` file contains your private Firebase credentials. Keep this file secure and never share or upload it to GitHub or any public repository.



3.4 Firestore Composite Indexes

Navigate to **Firestore Database** → **Indexes** → **Composite** and create these indexes:

Collection	Fields	Order
`tickets`	resultStatus ASC · drawType ASC	Ascending
`tickets`	gameType ASC · uploadedAt DESC	Mixed
`results_4d`	drawDate ASC · scrapedAt DESC	Mixed
`results_toto`	drawDate ASC · scrapedAt DESC	Mixed

ℹ️ If Firestore automatically prompts you to create a missing index (error in console with a direct link), you can click that link to create the index without manual steps.

4 Environment Configuration

4.1 Backend `.env` File

Create a file at `backend/.env` with the following content:

```
PORT=3001
GOOGLE_APPLICATION_CREDENTIALS=./serviceAccountKey.json
OCRSPACE_API_KEY=K87295079888957
JWT_SECRET=sglottery-secret-key-2026-change-in-production
```

Variable	Purpose
`PORT`	Port the Express server listens on (default 3001)
`GOOGLE_APPLICATION_CREDENTIALS`	Path to the Firebase service account JSON key
`OCRSPACE_API_KEY`	API key for OCR.Space (primary OCR engine); `helloworld` is a valid free demo key
`JWT_SECRET`	Secret for signing JWT tokens (change in production)

4.2 Mobile App — ngrok Backend URL

When testing on a physical device, `localhost` is not reachable from the phone. Use **ngrok** to create a public HTTPS tunnel to your local backend.

1. Install ngrok: `npm install -g ngrok` or download from https://ngrok.com/download
2. In a **separate terminal**, run: `npx ngrok http 3001`
3. Copy the HTTPS URL shown (e.g. `https://abc123.ngrok-free.app`)
4. Update this URL in all mobile source files listed below.

Files to update — change the `const API = '...'` line in each:

File	Variable
`mobile-app/app/(tabs)/upload.tsx`	`const API = 'https://YOUR-NGROK-URL/api/tickets'`
`mobile-app/app/(tabs)/history.tsx`	`const API = 'https://YOUR-NGROK-URL/api/tickets'`
`mobile-app/app/(tabs)/index.tsx`	`const API = 'https://YOUR-NGROK-URL/api/tickets'`
`mobile-app/app/(tabs)/results.tsx`	`const API = 'https://YOUR-NGROK-URL/api/results'`
`mobile-app/app/(tabs)/predict.tsx`	`const API = 'https://YOUR-NGROK-URL/api/predict'`
`mobile-app/hooks/useNotifications.ts`	`const API = 'https://YOUR-NGROK-URL/api/tickets'`

How to start ngrok:

Open a **new terminal window** (keep the backend terminal open) and type: `npx ngrok http 3001` — the Forwarding URL shown is your mobile backend URL.

The ngrok URL changes every time you restart ngrok. You must update all 6 files above whenever you get a new ngrok URL. The free plan provides a new random URL each session; paid plans offer static URLs.

5 Running the Applications

> You need **three separate terminal windows** — one for the backend, one for the web app, and one for the mobile app.

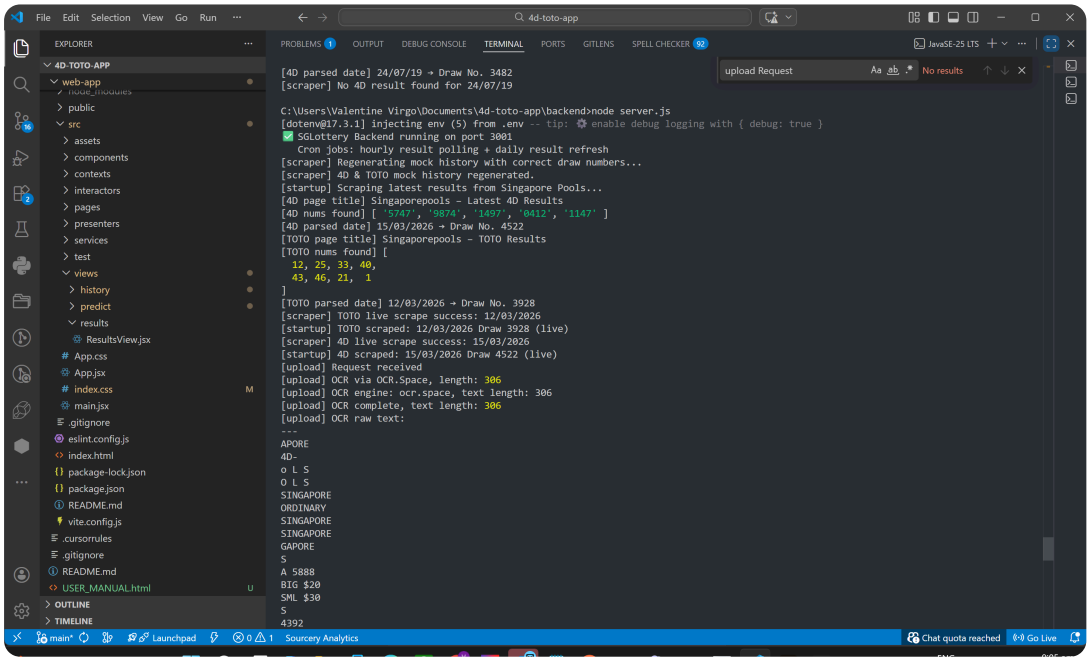
5.1 Start the Backend

```
cd backend
node server.js
```

Expected output:

```
✅ SGLottery Backend running on port 3001
   Cron jobs: hourly result polling + daily result refresh
```

Verify the backend is alive by opening **http://localhost:3001** in your browser — you should see a JSON health response.



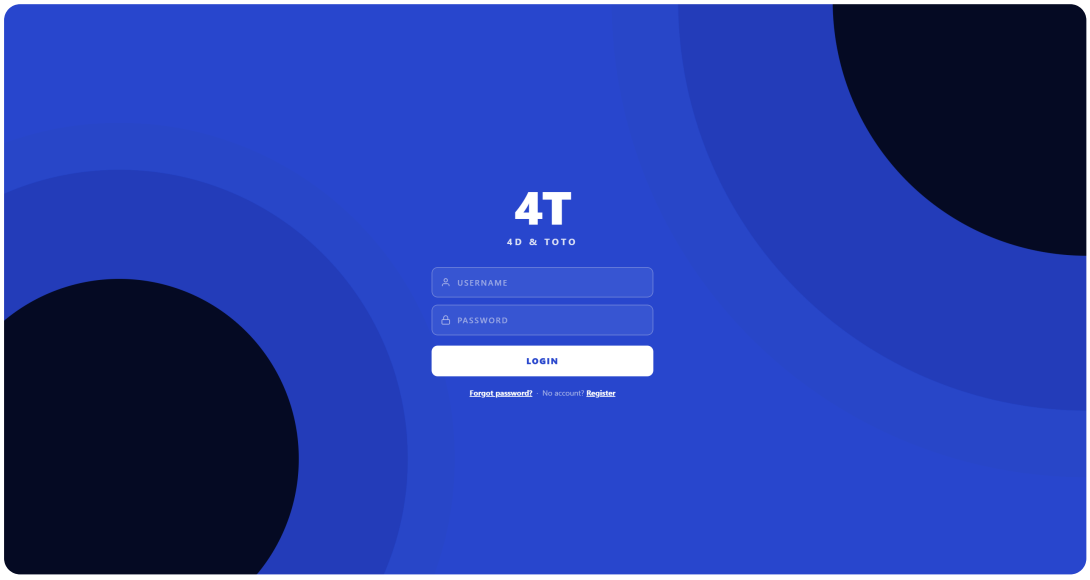
5.2 Start the Web App

```
cd web-app
npm run dev
```

Expected output:

```
VITE v7.x  ready in ~300ms
➜  Local:   http://localhost:5173/
```

Open **http://localhost:5173** in your browser. You will see the login screen — register a new account or log in to access the app.



5.3 Start the Mobile App

How to start the Mobile App:

Open a new
Command Prompt
window, then type the following commands one at a time:

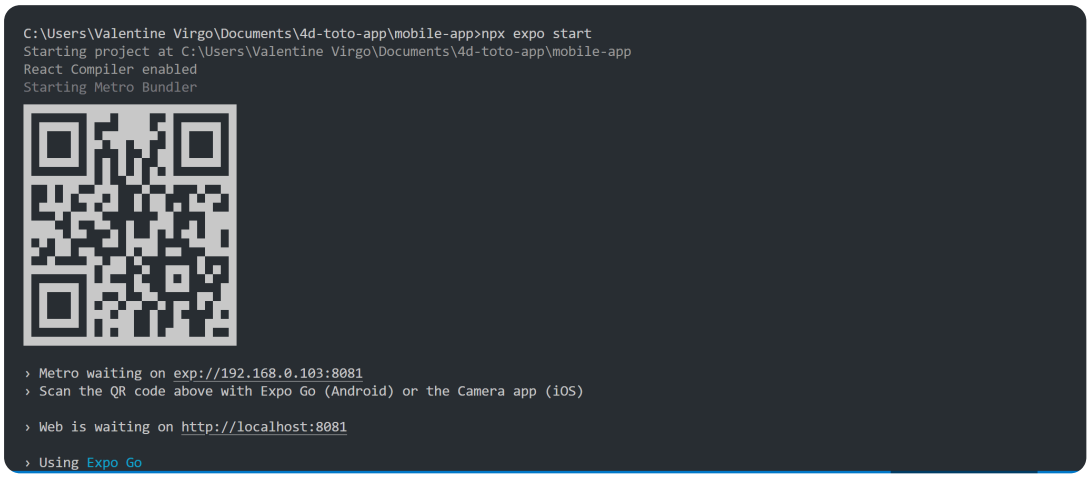
```
cd mobile-app
npx expo start
```

A QR code will appear in the terminal. Scan it with the
Expo Go
app on your phone to launch the mobile app.

```
cd mobile-app
npx expo start
```

Key	Action
a	Open Android emulator

`i`	Open iOS simulator (macOS only)
`w`	Open in web browser
Scan QR code	Open in **Expo Go** app on physical device



⚠️ **Port already in use?**

If port 3001 is blocked by a previous process, run:
`netstat -ano | findstr :3001` to find the PID, then `taskkill /PID <pid> /F`

6 Feature Guide

6.0 Login & Registration

The app requires an account. On first visit you will see the **"4T — 4D & TOTO"** login screen.

Registering a New Account

1. Click **No account? Register** at the bottom of the login form.
2. Enter your **Full Name**, **Email address**, and a **Password** (minimum 6 characters).
3. Click **Create Account**. You are logged in immediately.

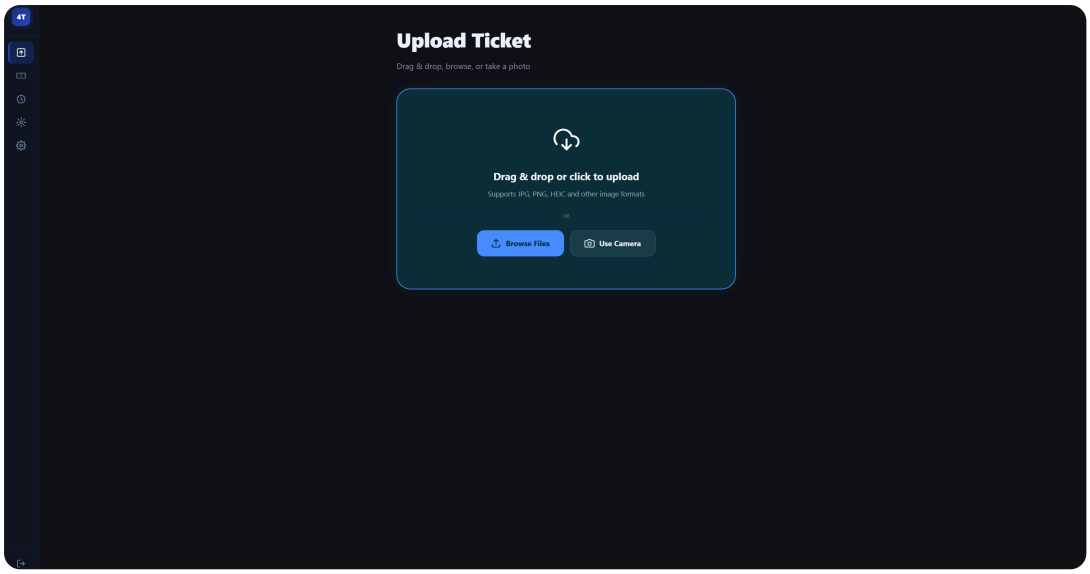
Logging In

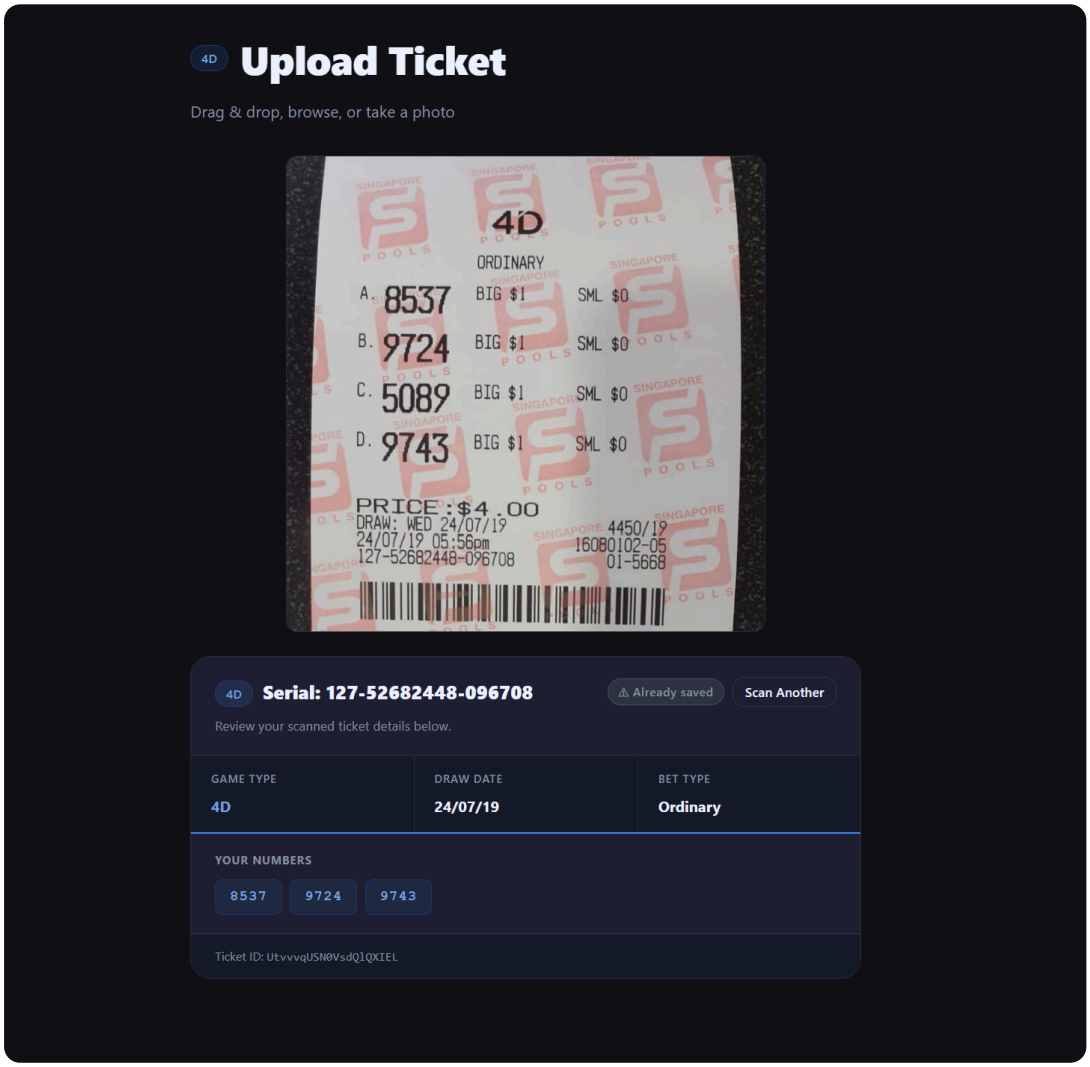
1. Enter your registered **Email** and **Password**.
2. Click **Login**. The main dashboard loads automatically.
3. Your session is saved in the browser — you will remain logged in on return visits.

> ℹ️ Authentication requires the backend to be running. If you see "Unable to create account" or "Account not found", ensure `node server.js` is running on port 3001.

6.1 Ticket Upload — Web

1. Click **Scan** in the navigation bar.
2. Drag and drop a ticket image onto the upload zone, *or* click **Browse Files** to select a file, *or* on mobile browsers click **Use Camera**.
3. A progress indicator appears while OCR processes the image (10–30 seconds).
4. The result card appears automatically showing all extracted data.
5. The ticket is saved to Firestore and visible in the History page.





💡 **Tip for best OCR results:**

> Use a clear, well-lit photo. Avoid shadows across the numbers. The ticket should be flat and fill most of the frame. The system automatically removes the Singapore Pools watermark before OCR processing.

6.2 Ticket Upload — Mobile App

1. Tap the **Scan** tab (📷 camera icon, centre of tab bar).
2. Tap **Take Photo** to use the device camera, *or* tap **Gallery** to pick an existing image.
3. Grant camera/gallery permission if prompted (first time only).
4. The app uploads the image to the backend and shows a loading spinner.
5. The OCR result card appears with all extracted fields.
6. A local push notification confirms the scan was successful.

SCREENSHOT 6.3

Mobile Scan tab — camera/gallery buttons and result card after successful OCR

6.3 OCR Extracted Data

Field	Example	Notes
Game Type	4D or TOTO	Auto-detected from ticket text
Bet Type	Ordinary · iBet · Roll · System N · Quick Pick · iTOTO	All Singapore Pools bet types supported
Draw Date	21/02/2026	Extracted from DRAW line on ticket
Numbers	5888, 4392	4-digit numbers (4D) or 6-number rows (TOTO)
Serial Number	846367-8-9785369-004	Extracted from line below the timestamp
Amount Paid	$10.00	From PRICE line on ticket

Combination Count	28	Number of bet combinations (C(N,6) for systems)

6.4 TOTO System Bet Expansion

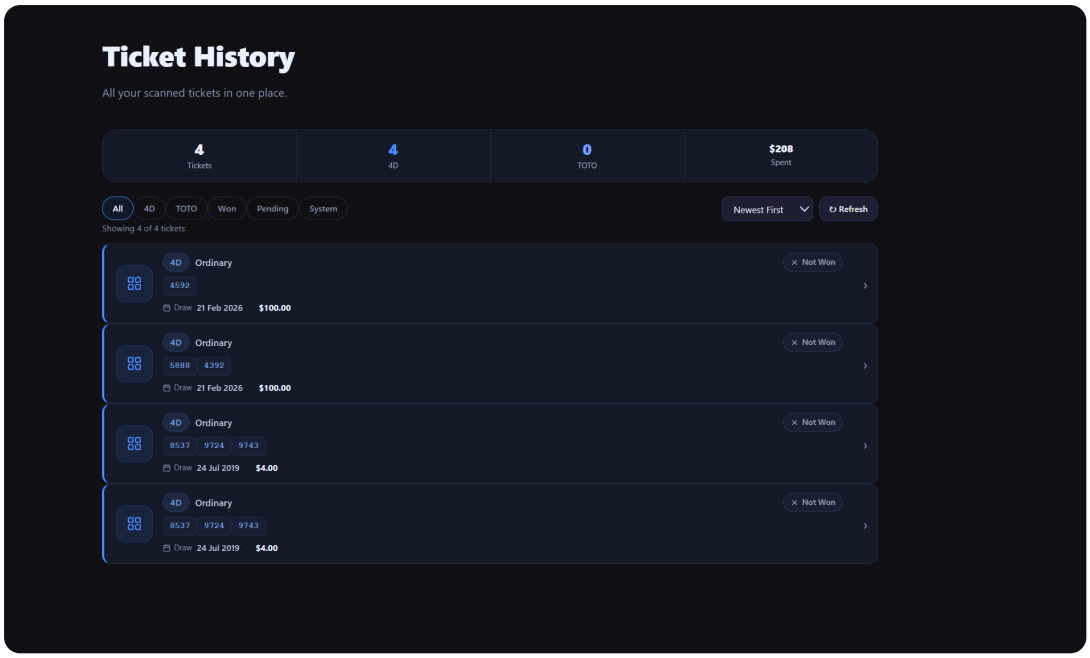
When a TOTO System ticket (System 7–12) is detected, the application automatically generates all possible 6-number combinations from the system numbers.

System	Numbers Selected	Combinations Generated
System 7	7	7
System 8	8	28
System 9	9	84
System 10	10	210
System 11	11	462
System 12	12	924

All combinations are stored in Firestore and used for result matching. Open any System ticket in the History detail modal to see the full expanded combination list.

6.5 Result Checking

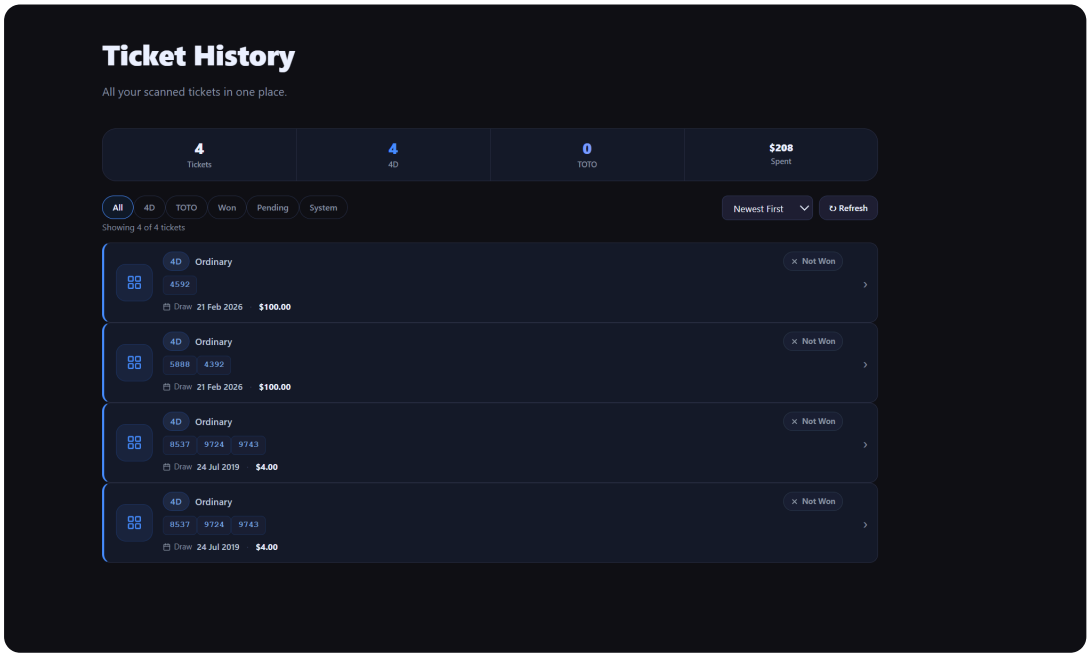
Scenario	Behaviour
Past draw (draw date already passed)	Results fetched immediately on upload. Win/loss status shown within seconds.
Future draw (draw date in the future)	Ticket saved as `Pending`. Backend cron job checks every hour.
Manual check	Click **Check My Ticket** on any history card, or open the detail modal and click **Check Now**.

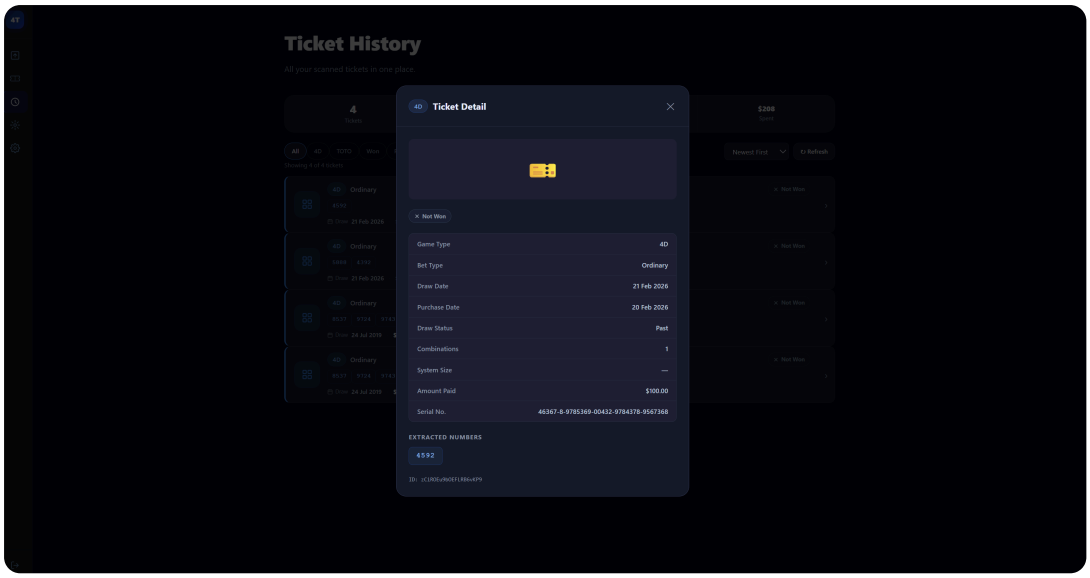


6.6 Ticket History

Web — Navigate to **History** in the navbar.

- Filter by: **All** **4D** **TOTO** **Won** **Pending** **System**
- Sort by: Newest · Oldest · Winning First · Amount ↓
- Click any card to open a detail modal with full OCR data, expanded combinations, and win matches.

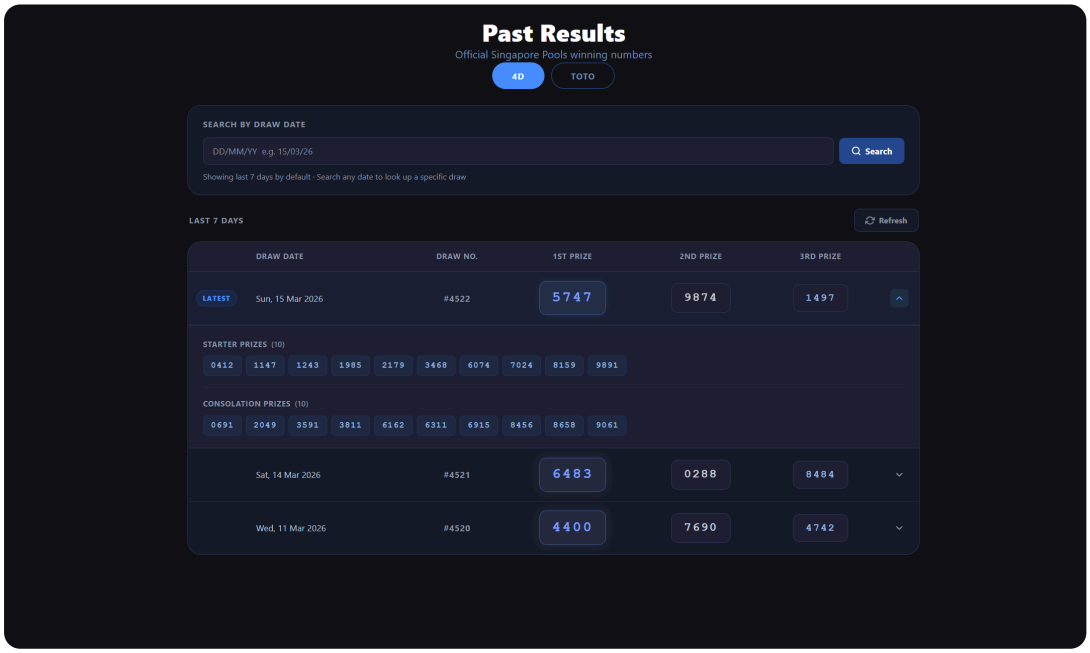


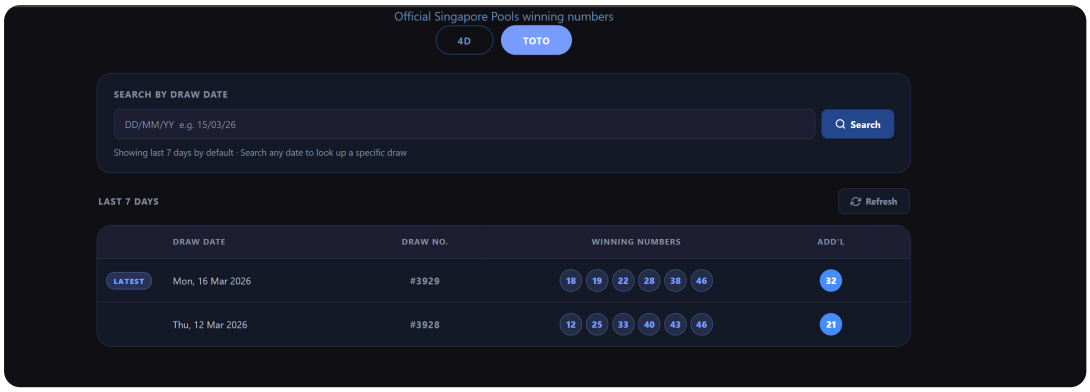


6.7 Past Results

Web — Navigate to **Results**. **Mobile** — Tap the Results tab (📊).

- Switch between **4D** and **TOTO** tabs.
- Click **Refresh Live** to re-scrape Singapore Pools (may take 5–10 seconds).
- **4D**: shows 1st/2nd/3rd prizes, Starter and Consolation numbers.
- **TOTO**: shows winning balls, additional number, and jackpot amount.

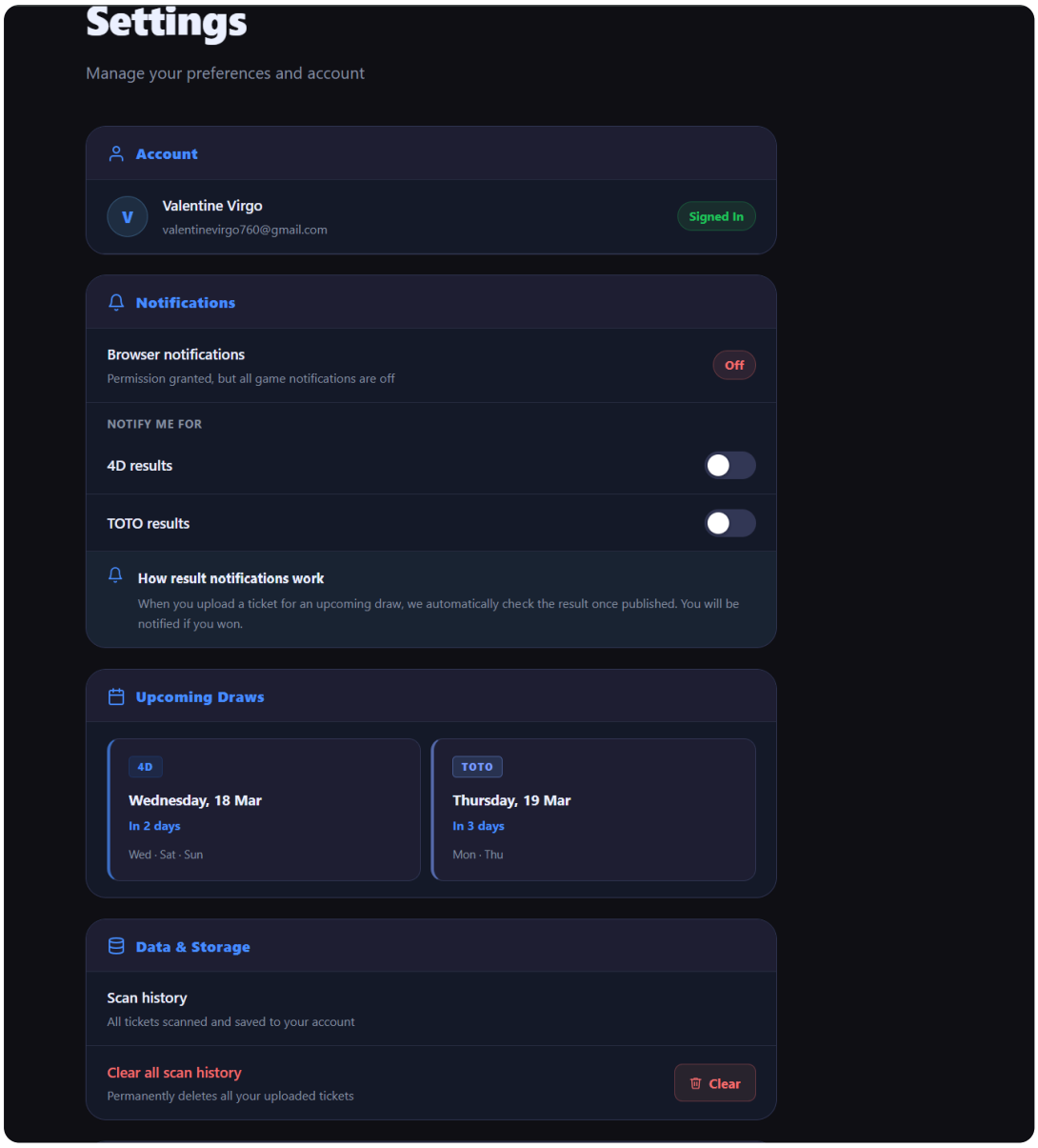




6.8 Push Notifications

Web: Browser push notification permission is requested on first visit. Once granted, notifications fire when a pending ticket's result becomes available. An in-app banner also appears in the top-right corner.

Mobile: Expo push notification permission is requested on first app launch. Local notifications fire when a pending ticket result is found during polling, and after every successful OCR scan.



6.9 Settings (Web)

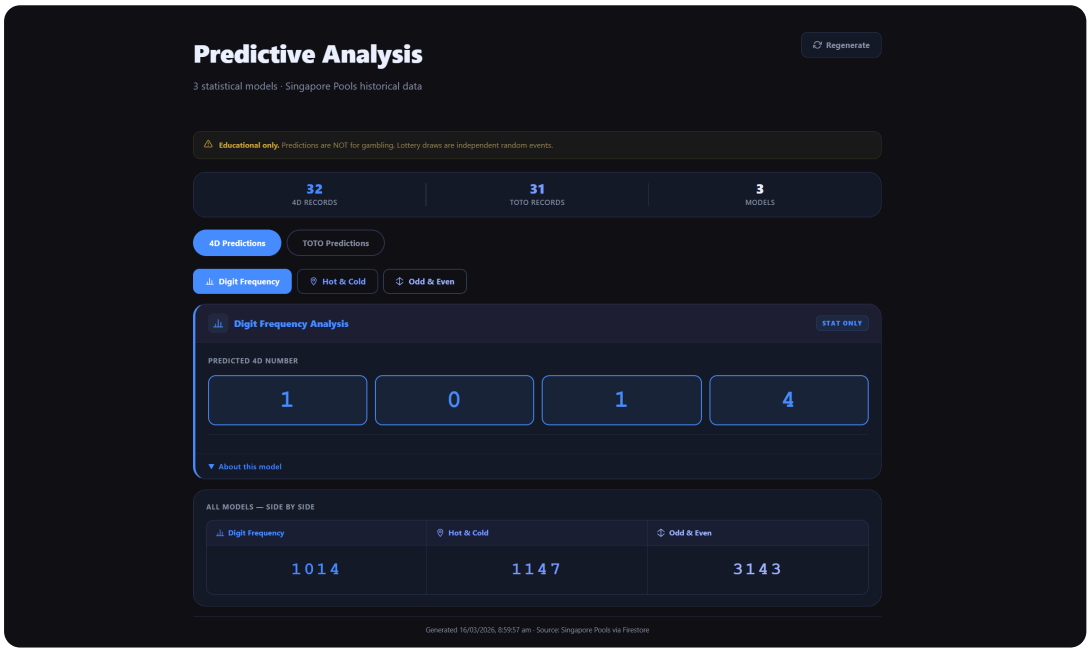
Navigate to **Settings** in the navbar.

- **Notifications**: Enable or disable browser notifications. Toggle 4D and TOTO game alerts separately. The badge shows **On** (green) when permission is granted and at least one game toggle is active.
- **Upcoming Draws**: Shows the next 4D and TOTO draw dates and times.
- **Notification Scheduler**: Schedules reminders before upcoming draw closings.

7 Predictive Analysis Guide

> ⚠️ **Disclaimer**
>
> All predictions are for **educational and entertainment purposes ONLY**. They are NOT intended for gambling or financial decisions. Lottery draws are statistically independent random events. Past patterns do not influence future draws.

Navigate to **Predict** (web navbar) or the **Predict** tab 🔮 (mobile).



The Three Models

Model 1 — Digit Frequency Analysis

Method	Counts how often each number/digit has appeared in recent draws, weighted by recency using exponential decay (recent draws count more).
4D Output	Selects the most frequently occurring digit for each of the 4 positions independently.
TOTO Output	Selects the 12 most frequently occurring numbers from 1–49 (System 12).
Confidence	~0.01% — equivalent to random chance. Demonstrates hot-number behaviour.

Model 2 — Hot & Cold Number Analysis

Method	Identifies numbers that have not appeared for longer than their statistically expected gap (cold numbers). Also tracks numbers on a recent streak (hot numbers).
4D Output	Selects the most overdue digit per position.
TOTO Output	Selects the 12 coldest (most overdue) numbers from 1–49.
Confidence	~0.01% — illustrates the Gambler's Fallacy: past absences do not increase future probability.

Model 3 — Odd & Even / Jackpot Pattern Analysis

Method	Analyses the ratio of odd vs even numbers in historical winning draws and the distribution of numbers across low/high ranges. Selects numbers matching the most common pattern.
4D Output	Picks digits matching the dominant odd/even positional pattern from past draws.
TOTO Output	Selects 12 numbers matching the most frequent odd/even distribution.
Confidence	~0.01–0.05% — slightly higher because pattern constraints reduce the search space.

Reading the TOTO System 12 Output

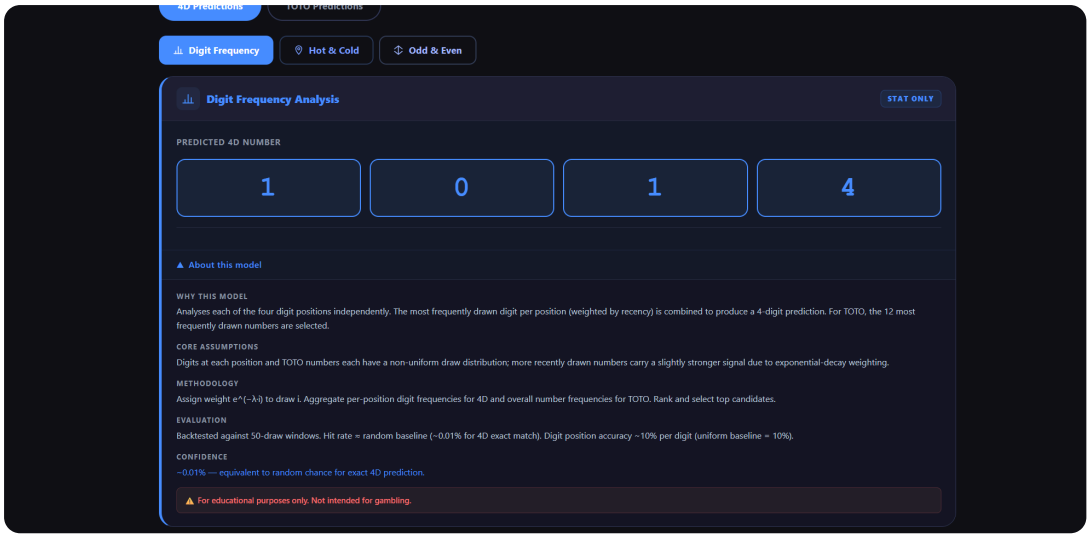
Each model outputs a System 12 TOTO prediction:

- **Primary (6 numbers)** — the "core" prediction, shown in blue.
- **Supplementary (6 numbers)** — the additional System 12 numbers, shown in lighter blue.
- Together these 12 numbers form a valid System 12 entry producing C(12,6) = 924 combinations.

Model Documentation Toggle

Click the ▼ **About this model** button at the bottom of each model card to expand the full documentation panel, which includes:

- **Why** this model was chosen
- **Assumptions** the model makes
- **Methodology** — how numbers are calculated
- **Evaluation** — how model performance is measured
- **Confidence** indicator
- **Disclaimer**



Regenerating Predictions

Click the **Regenerate** button on any model card to generate a new prediction set. Each run draws from the same historical data but applies randomised weighted sampling, so numbers will vary between runs.

8 Testing Each Feature

ℹ️ Ensure the backend is running on port 3001 before running any feature tests.

Test 1 — Ticket Upload & OCR

1. Start the backend (`node server.js`) and web app (`npm run dev`).
2. Open **http://localhost:5173/upload**.
3. Upload any clear photo of a Singapore Pools 4D or TOTO ticket.
4. **Expected:** Result card appears within 30 seconds with game type, draw date, numbers, bet type, and serial number populated.
5. Open **Firebase Console** → **Firestore** → **tickets** collection and verify a new document was created.

Test 2 — TOTO System Bet Expansion

1. Upload a TOTO System 7 or higher ticket photo.
2. In the result card, verify `expandedCombinationCount` is shown (e.g. 28 for System 8).
3. Open the ticket in **History** → **click card** → **detail modal**.
4. **Expected:** The "Expanded Combinations" section lists all combinations with a count (e.g. "28 total").

Test 3 — Past Results

1. Navigate to **http://localhost:5173/results**.
2. Click **Refresh Live**.
3. **Expected:** 4D prize numbers (1st, 2nd, 3rd, Starters, Consolation) and TOTO winning balls load within 10 seconds.

4 If Singapore Pools blocks scraping, mock demo data is shown — this is expected behaviour, labelled "Demo data".

Test 4 — Result Checking (Past Draw)

1 Upload a ticket with a **past draw date** (draw date before today).

2 **Expected:** The result card immediately shows either `Won` or `Not Won` — no manual action needed.

3 Check the ticket in History — the same status badge should appear on the card.

Test 5 — Result Checking (Future Draw)

1 Upload a ticket with a **future draw date**.

2 **Expected:** The result card shows `Pending` .

3 To manually trigger a check: call `POST http://localhost:3001/api/results/check/:ticketId` using a tool like Postman or curl.

4 Alternatively, wait for the hourly cron job to run automatically.

Test 6 — Predictive Analysis

1 Navigate to **http://localhost:5173/predict**.

2 **Expected:** Three model cards load, each showing a 4D predicted number and 12 TOTO balls.

3 Click ▼ **About this model** — verify the documentation panel expands with all fields (why, assumptions, methodology, evaluation, confidence, disclaimer).

4 Click **Regenerate** — numbers should change with each click.

5 Switch between **4D** and **TOTO** tabs — both should display different prediction formats.

Test 7 — Push Notifications (Web)

1 Open the web app. If not yet prompted, navigate to **Settings** and click **Enable Notifications**.

2 Allow the browser notification permission when the browser dialog appears.

3 Manually trigger a result check for a past ticket via the API or click **Check My Ticket** on a history card.

4 **Expected:** A browser push notification fires with the ticket outcome.

Test 8 — Mobile Ticket Scan

1 Start ngrok: `npx ngrok http 3001` and update all mobile API URLs.

2 Run `npx expo start` and scan the QR code with Expo Go.

3 Tap the **Scan** tab → **Take Photo** → photograph a lottery ticket.

4 **Expected:** Result card appears in the app and a local push notification fires.

5 Tap **History** tab — the new ticket should appear at the top.

9 Troubleshooting

"Upload failed. Make sure backend is running."

- Verify the backend is running: open `http://localhost:3001` in the browser.
- Check for CORS errors in the browser developer console (F12 → Console tab).
- For mobile: ensure the ngrok URL is correct, has been updated in all source files, and that ngrok is currently running.

OCR Returns No Numbers / Wrong Numbers

- The image may be blurry, low contrast, or at an angle. Use a flat, well-lit photo.
- Check the backend terminal for `[upload] OCR raw text:` log to see exactly what OCR extracted.
- The system uses OCR.Space as the primary engine and Tesseract as fallback — check for API limit errors in the terminal.
- If using the free OCR.Space key (`helloworld`), it has a 25,000 requests/month limit.

"Could not load tickets" (History page)

- The backend must be running and reachable.
- Check that `serviceAccountKey.json` exists in the `backend/` folder.
- Verify Firestore Database is enabled in Firebase Console (not just created — must be in Native mode).
- Open browser Developer Tools → Network tab → look for the failing API request and inspect the error response.

Past Results Show "Demo Data"

- This is expected and by design. Singapore Pools may block automated scraping (bot detection).
- Mock data is automatically served as a fallback, clearly labelled "Demo data".
- For production use, a licensed Singapore Pools data feed or paid proxy service would be required.

Cron Job Not Running / Results Not Auto-Updating

- The backend process must stay running continuously — do not close the terminal.
- The cron runs every hour on the backend. To test immediately, call:
  `POST http://localhost:3001/api/results/check/:ticketId`

Expo Push Notifications Not Received

- Check that notification permissions are granted in the device Settings app.
- Physical device required — simulators and emulators may not receive push notifications.
- Expo Go has limited background push support; a standalone Expo build is required for full production push.
- Polling happens every 60 seconds in the foreground — notifications appear when the app is open.

Port 3001 Already in Use

```
# Windows — find and kill the process holding port 3001
netstat -ano | findstr :3001
taskkill /PID <pid> /F

# Then restart the backend
node server.js
```

Mobile App Cannot Connect to Backend

- Physical devices cannot reach `localhost` — ngrok is required.
- Ensure the ngrok tunnel is active (check the ngrok terminal window).
- Confirm the ngrok HTTPS URL is updated in all 6 mobile source files.
- The `ngrok-skip-browser-warning: true` header is already set in all mobile API calls — this is required for ngrok free tier.

10 Known Issues

Issue	Status	Impact	Workaround
Singapore Pools scraper may be blocked by bot detection	**Known**	Past Results shows demo data instead of live data	Mock data fallback is automatic and labelled clearly
Firebase Storage disabled (requires Blaze plan)	**Known**	Ticket history cards show game-type icon instead of photo thumbnail	All features work fully; enable Storage by upgrading Firebase plan
OCR accuracy varies with image quality	**Known**	Numbers may be missed or misread on poor-quality scans	Use high-resolution, well-lit, flat photos
Expo background polling limited in Expo Go	**Known**	Push notifications only received while app is open	Build a standalone Expo app for full background push support
TOTO System 12 expansion (924 combos) write is async	**Minor**	Brief delay before all combinations appear in History detail modal	Wait a few seconds and refresh the History page
ngrok URL changes each session (free plan)	**Known**	Mobile app loses connection to backend after ngrok restart	Update the 6 API URL constants in mobile source files; or use ngrok paid plan for a static URL

11 Data Privacy & Security

What Data Is Collected

Data	Where Stored	Retention
Extracted ticket data (numbers, dates, bet type)	Firebase Firestore	Until deleted by user
Ticket serial numbers	Firebase Firestore	Until deleted by user
OCR raw text (for debugging)	Firebase Firestore	Until deleted by user
Device push token	Backend memory only (not persisted)	Session only

What Is NOT Collected

- Personal identity information (name, NRIC, email, phone number)
- Financial account or payment details
- Device location data
- Ticket images (Firebase Storage disabled in this build)

Security Practices

- `serviceAccountKey.json` is excluded from Git via `.gitignore`.
- The `.env` file is excluded from Git — API keys are never committed.
- The ngrok URL changes every session — never commit it to source control.
- All API calls from the mobile app use HTTPS (via ngrok).
- JWT-based authentication middleware is implemented for protected endpoints.

Recommended Firestore Rules (Production)

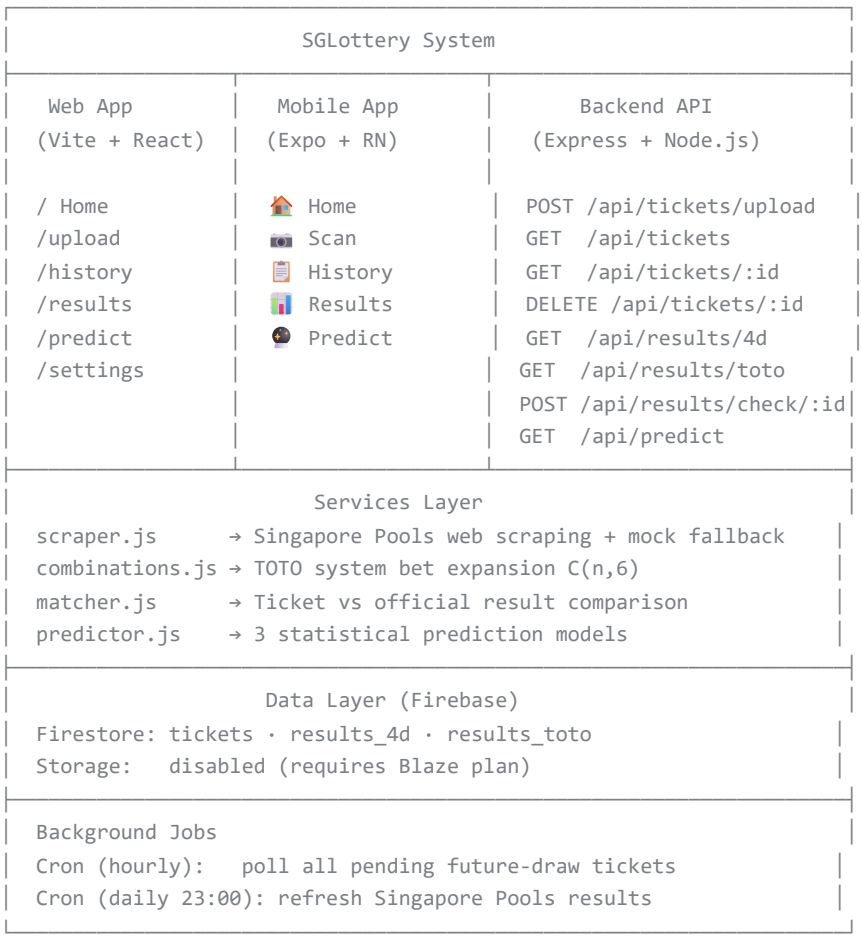
```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /tickets/{ticket} {
      allow read, write: if request.auth != null;
    }
    match /results_4d/{doc} {
      allow read: if true;
      allow write: if request.auth != null;
    }
```

```
    match /results_toto/{doc} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```

Ethics Statement

⚠️ This application is built for **educational and personal tracking purposes only**. It does not encourage, facilitate, or promote gambling. All predictive analysis features include prominent disclaimers stating that lottery draws are random and predictions carry no predictive validity. The application does not provide odds calculations, financial advice, or any guarantee of winning outcomes.

12 Architecture Overview



Technology Stack

Layer	Technology	Version	Purpose
Backend	Node.js + Express	v24 / v5	REST API, OCR processing, cron jobs
OCR (Primary)	OCR.Space API	v2	Cloud OCR with table/text detection
OCR (Fallback)	Tesseract.js	5.x	Local OCR with watermark removal
Image Processing	Sharp	0.33	Watermark removal, resizing, greyscale
Database	Firebase Firestore	Admin SDK 13	NoSQL document storage

Web Frontend	React 19 + Vite	19 / 7	SPA with React Router v7
Mobile Frontend	Expo SDK 54 + React Native	0.76	Cross-platform iOS/Android app
Routing (Mobile)	expo-router	4.x	File-based routing for Expo
Notifications (Web)	Browser Notification API	—	Push notifications in browser
Notifications (Mobile)	expo-notifications	0.29	Local + remote push notifications

Folder Structure

```
4d-toto-app/
├── backend/
│   ├── server.js              ← Express entry point + cron jobs
│   ├── firebase.js            ← Firebase Admin SDK init
│   ├── routes/
│   │   ├── tickets.js         ← Upload, list, detail, delete endpoints
│   │   ├── results.js         ← 4D/TOTO results scraping endpoints
│   │   └── predict.js         ← Prediction model endpoints
│   ├── services/
│   │   ├── scraper.js         ← Singapore Pools scraper + fallback
│   │   ├── combinations.js    ← C(n,6) system bet expansion
│   │   ├── matcher.js         ← Ticket vs result comparison
│   │   └── predictor.js       ← 3 statistical models
│   ├── middleware/auth.js      ← JWT auth middleware
│   ├── serviceAccountKey.json ← Firebase key (git-ignored)
│   └── .env                   ← Environment variables (git-ignored)
│
├── web-app/
│   └── src/
│       ├── App.jsx            ← Router + Navbar + Footer
│       ├── index.css          ← Full design system CSS
│       ├── pages/             ← Page wrappers
│       ├── views/             ← Feature view components
│       │   ├── history/       ← Ticket history + detail modal
│       │   ├── predict/       ← Prediction model cards
│       │   ├── results/       ← Past results browser
│       │   └── upload/        ← Upload + OCR result card
│       ├── presenters/        ← Business logic (MVP pattern)
│       ├── services/          ← API client + notification scheduler
│       └── components/        ← Shared UI components
│
├── mobile-app/
│   └── app/(tabs)/
│       ├── index.tsx          ← Home tab
│       ├── upload.tsx         ← Scan tab
│       ├── history.tsx        ← History tab
│       ├── results.tsx        ← Results tab
│       └── predict.tsx        ← Predict tab
│
├── USER_MANUAL.md             ← Source manual (Markdown)
└── USER_MANUAL.html           ← This document
```

13 API Reference

Base URL: `http://localhost:3001`

POST /api/tickets/upload

Upload a ticket image for OCR processing and result checking.

Property	Value
Content-Type	`multipart/form-data`
Field name	`ticket` (image file)

Response (200):

```
{
  "success": true,
  "ticketId": "abc123",
  "gameType": "TOTO",
  "drawDate": "06/03/2026",
  "numbers": ["05  15  22  33  41  46"],
  "betType": "System 8",
  "systemSize": 8,
  "combinationCount": 28,
  "expandedCombinationCount": 28,
  "drawType": "past",
  "comparison": { "won": false, "matches": [], "prizeTier": null }
}
```

GET /api/tickets

List all tickets.

Query Param	Type	Description
`gameType`	string	Filter by `4D` or `TOTO`
`limit`	number	Max results (default 50)

GET /api/tickets/:id

Get a single ticket by Firestore document ID.

DELETE /api/tickets/:id

Delete a ticket. Requires authentication header.

POST /api/results/check/:ticketId

Manually trigger result checking for a single ticket. Fetches the draw result and updates `resultStatus`, `prizeTier`, and `winMatches` in Firestore.

GET /api/results/4d

Get past 4D draw results.

Query Param	Type	Description
`date`	string	Filter by date DD/MM/YYYY
`limit`	number	Max results (default 20)

GET /api/results/toto

Get past TOTO draw results. Same query params as 4D.

GET /api/predict

Generate predictions from all 3 models.


```
{
  "predictions": [
    {
      "model": "Digit Frequency Analysis",
      "modelId": "frequency",
      "predicted4D": "3829",
      "predictedTOTO": [4, 7, 15, 22, 33, 36, 41, 44, 46, 47, 48, 49],
      "confidenceScore": 0.12,
      "why": "...",
      "assumptions": "...",
      "methodology": "...",
      "evaluation": "...",
      "confidence": "...",
      "disclaimer": "..."
    }
  ],
  "dataPoints": { "fourd": 50, "toto": 100 },
  "generatedAt": "2026-03-16T10:00:00.000Z",
  "disclaimer": "For educational purposes only."
}
```

14 Changelog & Development History

V2.0 March 2026

Complete UI Redesign + Notification System + Prediction Model Documentation

- Redesigned colour system: grey/white dominant with blue (`#4a8fff`) as the single accent colour — removed all red, gold, cyan and purple accents for a cleaner, more elegant look.
- Fixed notification status badge: now correctly shows **On** (green) when permission is granted and at least one game toggle is active, and **Off** (red) when all game toggles are disabled.
- Fixed notification scheduler: `checkAndNotify()` is now called at timer execution instead of deprecated `fire()`.
- Added expandable "About this model" documentation panel to each prediction model card — shows why, assumptions, methodology, evaluation, confidence, and disclaimer.
- Added ticket thumbnail support in History cards (shows actual photo when available via Firebase Storage; falls back to game-type icon).
- Fixed serial number OCR extraction: now correctly reads the serial number from the line immediately after the AM/PM timestamp, ignoring time values.
- Disabled Firebase Storage image upload (requires Blaze plan) — silently removed from upload flow.
- Changed Android notification light colour from red to blue (`#4a8fff`).
- Updated User Manual model names to match actual implementation: Digit Frequency Analysis · Hot & Cold Number Analysis · Odd & Even / Jackpot Pattern Analysis.

V1.5 February 2026

Mobile App + OCR Improvements + Push Notifications

- Built full mobile app (Expo SDK 54) with 5-tab layout: Home · Scan · History · Results · Predict.
- Integrated OCR.Space API as primary OCR engine with Tesseract.js as fallback.
- Implemented Singapore Pools watermark removal using HSV colour-space processing — saturated red/pink pixels replaced with white before OCR, preserving black ticket text.
- Added expo-notifications push notification system with 60-second polling for pending ticket results.
- Implemented deduplication: identical tickets (same fingerprint) are rejected on re-upload.
- Added JWT authentication middleware for protected API endpoints.
- Upgraded TOTO System Bet expansion to handle all System 7–12 combinations (up to 924 for System 12).

V1.2 January 2026

Predictive Analysis + Past Results Browser

- Implemented three statistical prediction models: Digit Frequency, Hot & Cold, Odd & Even Pattern.
- Predictions return TOTO System 12 format (12 numbers, C(12,6) = 924 combinations).
- Each model returns full documentation fields: why, assumptions, methodology, evaluation, confidence, disclaimer.
- Built Singapore Pools scraper for 4D and TOTO past results with mock data fallback.
- Added daily cron job (23:00) to refresh result data from Singapore Pools.
- Built Past Results view for web and mobile with live refresh capability.

V1.1 December 2025

Result Checking + History + Web App

- Built full web app (Vite + React 19 + React Router v7) with 5 pages: Home · Scan · History · Results · Predict · Settings.
- Implemented automated result checking against Singapore Pools for past-draw tickets.
- Added hourly cron job to poll pending future-draw tickets.
- Built History page with filter tabs (All/4D/TOTO/Won/Pending/System), sort options, stats bar, and detail modal.
- Added browser push notification system (Web Notification API + localStorage scheduler).
- Implemented Settings page with notification toggles, upcoming draw calendar, and schedule management.

V1.0 November 2025

Initial Release — Core Upload + OCR + Firestore

- Express.js backend with Tesseract OCR and Firebase Admin SDK.
- POST /api/tickets/upload — accepts ticket image, runs OCR, saves to Firestore.
- Supports 4D (Ordinary, iBet, Roll, System 5/6/7) and TOTO (Standard, System 7–12, Quick Pick) ticket types.
- Extracts: game type, draw date, purchase date, numbers, bet type, system size, serial number, amount paid.
- Drag-and-drop upload zone on web with OCR result card display.
- GET /api/tickets — list all tickets with deduplication and filtering.
- Firebase Firestore as primary data store with service account key authentication.